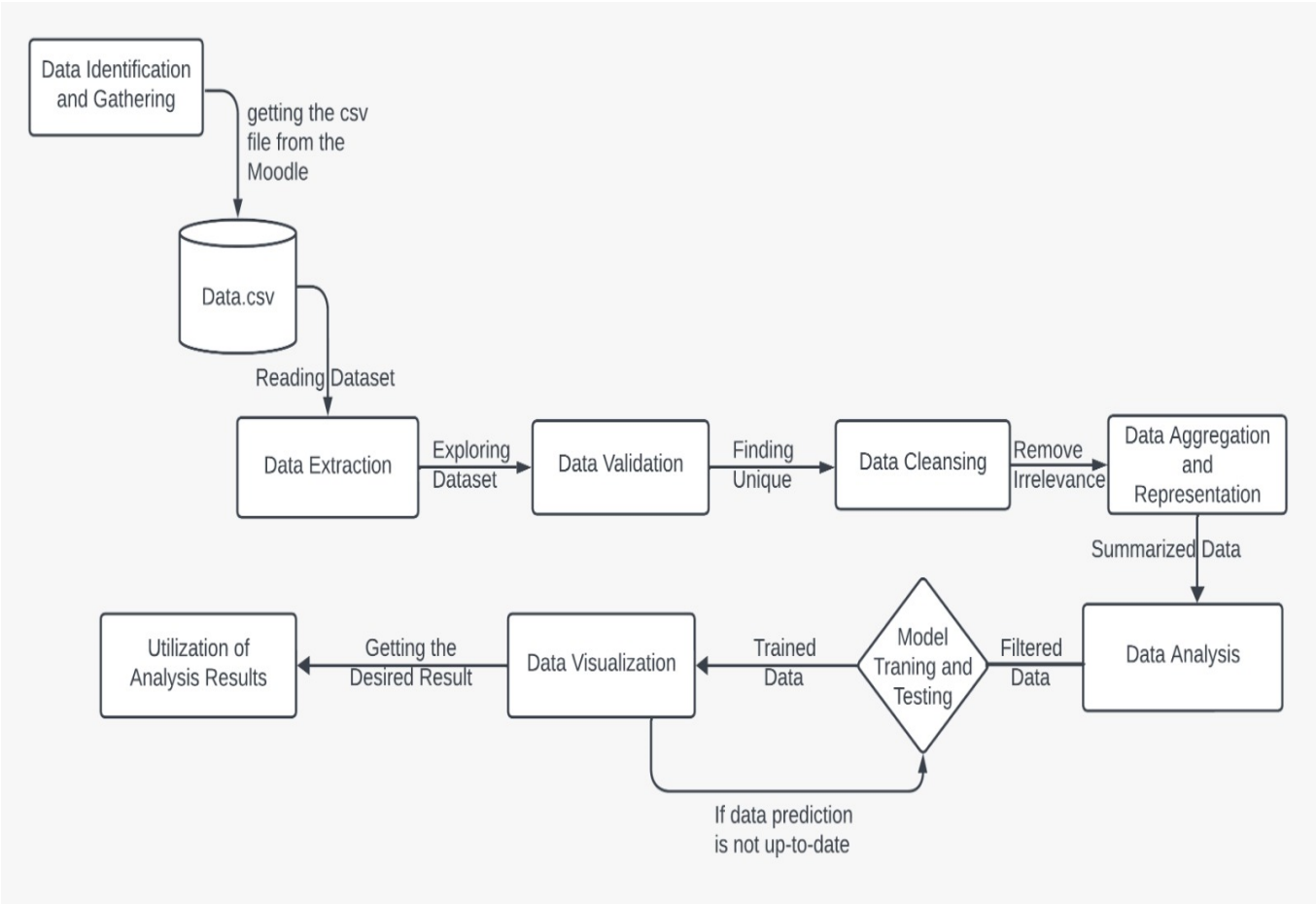
➢ Flow chart

➔ Nine stages of Big Data analytics

- The flow chart shows the nine stages of big data analytics in detail sequence wise.



1. Data Loading and Extraction: Loading and extracting the dataset.

1.1. Importing Libraries: Importing the necessary libraries for data pre-processing and analysis such as Pandas, Numpy, MatPlotLib, and Seaborn.

Importing the Libraries

```
[1]: import pandas as pd
     import numpy as np
     import seaborn as sns
     import matplotlib.pyplot as plt
```

Figure 2.1

1.2. Data Extraction: Extract and take a first look at the dataset.

1.2.1. Pandas.read_csv(): Reads the given dataset in a DataFrame.

Reading the data from a csv file

```
[3]: data = pd.read_csv('dataset.csv')
```

1.2.2. DataFrame.head() and DataFrame.tail(): Displays a few rows for exploring the dataset.

Looking at the first five rows of the dataset

```
[6]: data.head()
```

[6]:

	symbol	exchange	date	adjusted close	option symbol	expiration	strike	call/put	style	ask	...	iv	volume	open interest	stock price for iv	*	delta	vega	gamma
0	SPY	NYSEArca	07/30/2021	438.51	SPY 210730C00215000	07/30/2021	215.0	C	A	224.56	...	-1.0	0	0	438.97	NaN	0.0	0.0	0.0
1	SPY	NYSEArca	07/30/2021	438.51	SPY 210730P00215000	07/30/2021	215.0	P	A	0.01	...	-1.0	0	1401	438.97	NaN	0.0	0.0	0.0
2	SPY	NYSEArca	07/30/2021	438.51	SPY 210730C00220000	07/30/2021	220.0	C	A	219.56	...	-1.0	0	1	438.97	NaN	0.0	0.0	0.0
3	SPY	NYSEArca	07/30/2021	438.51	SPY 210730P00220000	07/30/2021	220.0	P	A	0.01	...	-1.0	50	328	438.97	NaN	0.0	0.0	0.0
4	SPY	NYSEArca	07/30/2021	438.51	SPY 210730C00225000	07/30/2021	225.0	C	A	214.56	...	-1.0	0	0	438.97	NaN	0.0	0.0	0.0

5 rows × 22 columns

Looking at the last five rows of the dataset

```
[7]: data.tail()
```

[7]:

	symbol	exchange	date	adjusted close	option symbol	expiration	strike	call/put	style	ask	...	iv	volume	open interest	stock price for iv	*	delta
221041	SPY	NYSEArca	07/01/2021	430.43	SPY 231215P00630000	12/15/2023	630.0	P	A	206.50	...	0.148305	0	1	430.26	*	-0.924500
221042	SPY	NYSEArca	07/01/2021	430.43	SPY 231215C00635000	12/15/2023	635.0	C	A	2.20	...	0.148310	0	146	430.26	NaN	0.051628
221043	SPY	NYSEArca	07/01/2021	430.43	SPY 231215P00635000	12/15/2023	635.0	P	A	211.50	...	0.148310	0	0	430.26	*	-0.928176
221044	SPY	NYSEArca	07/01/2021	430.43	SPY 231215C00640000	12/15/2023	640.0	C	A	2.73	...	0.148310	5	1	430.26	*	0.048170
221045	SPY	NYSEArca	07/01/2021	430.43	SPY 231215P00640000	12/15/2023	640.0	P	A	216.50	...	0.148310	0	0	430.26	*	-0.931634

5 rows × 22 columns

2.Data Validation and Cleansing:

2.1. **Data Validation:** Validate the dataset.

2.1.1. **DataFrame.columns:** List of columns to identify features.

All the columns of our dataset

```
[8]: data.columns

[8]: Index(['symbol', 'exchange', 'date', 'adjusted close', 'option symbol',
            'expiration', 'strike', 'call/put', 'style', 'ask', 'bid', 'mean price',
            'iv', 'volume', 'open interest', 'stock price for iv', '*', 'delta',
            'vega', 'gamma', 'theta', 'rho'],
           dtype='object')
```

2.1.2. DataFrame.shape: Finding volume of dataset.

No.of rows and columns in our dataset

```
[9]: data.shape

[9]: (221046, 22)
```

2.1.3. DataFrame.describe(): Describing various mathematical and statistical values in the dataset.

Generating descriptive statistics to summarize central tendency,dispersion,shape of the dataset

```
[10]: data.describe()
```

[10]:

	adjusted close	strike	ask	bid	mean price	iv	volume	open interest	stock price for iv	delta	
count	221046.000000	221046.000000	221046.000000	221046.000000	221046.000000	221046.000000	221046.000000	221046.000000	221046.000000	221046.000000	221046.0
mean	435.089149	398.702076	41.667365	40.967420	41.315752	0.234593	405.715661	1660.743542	434.966436	0.102498	0.5
std	3.863763	93.725546	60.504762	59.876656	60.189057	0.225324	5100.355521	6040.663628	3.837734	0.614277	0.5
min	424.970000	25.000000	0.010000	0.000000	0.000000	-1.000000	0.000000	0.000000	425.020000	-1.000000	0.0
25%	432.930000	349.000000	1.660000	1.500000	1.600000	0.139646	0.000000	1.000000	432.520000	-0.240680	0.0
50%	435.460000	411.000000	15.910000	15.440000	15.650000	0.206940	0.000000	100.000000	435.440000	0.000000	0.2
75%	438.510000	454.000000	52.550000	51.550000	52.040000	0.306208	19.000000	918.000000	438.140000	0.731058	0.8
max	441.020000	655.000000	416.890000	415.230000	416.060000	1.805012	446333.000000	169905.000000	441.050000	1.000000	2.6

2.1.4. DataFrame.isnull(): Representing the missing values in the dataset.

Representing the missing values in the Dataset

```
[11]: data.isnull().sum()

[11]: symbol                  0
      exchange                0
      date                    0
      adjusted close          0
      option symbol           0
      expiration              0
      strike                  0
      call/put                0
      style                   0
      ask                     0
      bid                     0
      mean price              0
      iv                      0
      volume                  0
      open interest           0
      stock price for iv      0
      *                  192007
      delta                   0
      vega                    0
      gamma                   0
      theta                   0
      rho                     0
      dtype: int64
```

2.1.5. DataFrame.nunique(): Finding the unique values for each column.

Finding the unique values for each columns

```
[10]: data.nunique()
```

```
[10]: symbol                1
      exchange              1
      date                 21
      adjusted close       21
      option symbol     13960
      expiration           48
      strike              312
      call/put              2
      style                 1
      ask               22926
      bid               22611
      mean price        36172
      iv               146910
      volume             6205
      open interest     14825
      stock price for iv   21
      *                     1
      delta            182814
      vega             179397
      gamma             28374
      theta             91796
      rho              183742
      dtype: int64
```

2.2. Data Cleansing: Filter, clean, and refine the dataset.

2.2.1. DataFrame.drop(): Drop the unnecessary fields.

Dropping some of the rows and columns

```
[4]: data=data.drop(['symbol', 'exchange','option symbol','*','style'], axis=1)
```

2.2.2. Filtering dataset for further evaluation: Fetching the data until 29th date.

Fetching the Data until 29th date.

```
[5]: data['date'] = pd.to_datetime(data['date'])
     data=data.rename(columns = {'stock price for iv':'stockPrice'})
     data_till_29 = data[~(data['date'] > '2021-07-29')]S
     data_till_29.head()
```

[5]:

	date	adjusted close	expiration	strike	call/put	ask	bid	mean price	iv	volume	open interest	stockPrice	delta	vega	gamma	theta	rho
10726	2021-07-29	440.65	07/30/2021	215.0	C	225.46	224.71	225.085	0.290659	0	0	440.13	0.999908	0.0	0.0	0.0	0.005890
10727	2021-07-29	440.65	07/30/2021	215.0	P	0.01	0.00	0.000	0.325459	0	1401	440.13	0.000000	0.0	0.0	0.0	0.000000
10728	2021-07-29	440.65	07/30/2021	220.0	C	220.46	219.71	220.085	0.290659	0	1	440.13	0.999908	0.0	0.0	0.0	0.006027
10729	2021-07-29	440.65	07/30/2021	220.0	P	0.01	0.00	0.000	0.325459	0	328	440.13	0.000000	0.0	0.0	0.0	0.000000
10730	2021-07-29	440.65	07/30/2021	225.0	C	215.46	214.71	215.085	0.290659	0	0	440.13	0.999908	0.0	0.0	0.0	0.006164

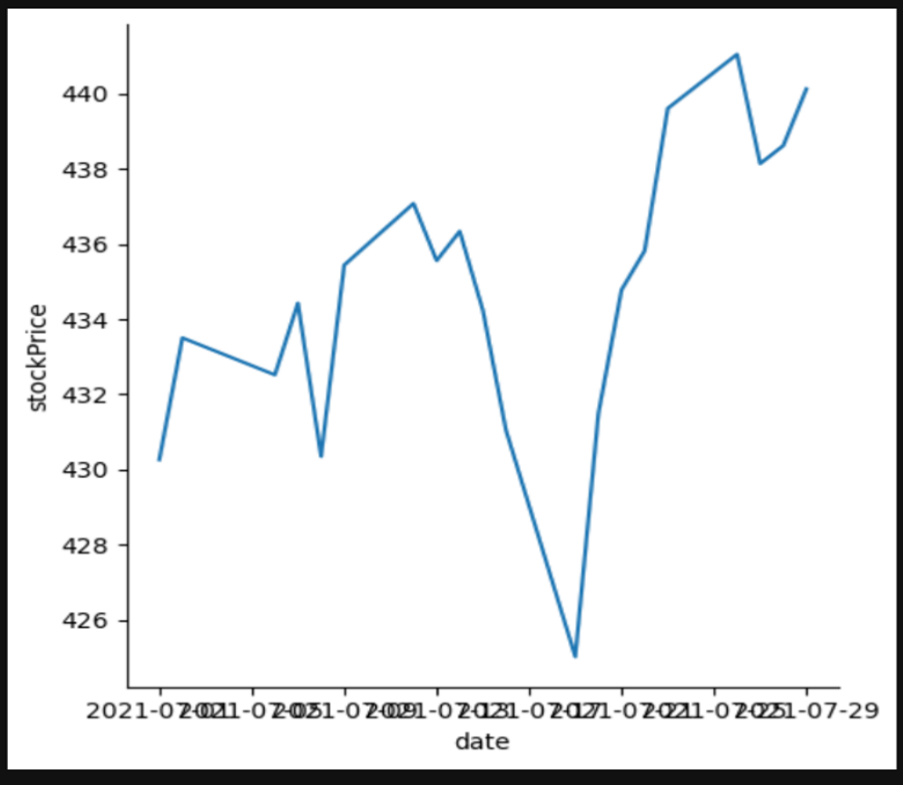
3.Data Representation:

3.1. Date vs Stock Price: Plotting the Date vs Stock Price Graph using relplot() function of seaborn.

Plotting the Date vs Stock price graph

```
[39]: sns.relplot( x="date", y="stockPrice",data=data_till_29,kind='line')

[39]: <seaborn.axisgrid.FacetGrid at 0x199dffb5ff0>
```

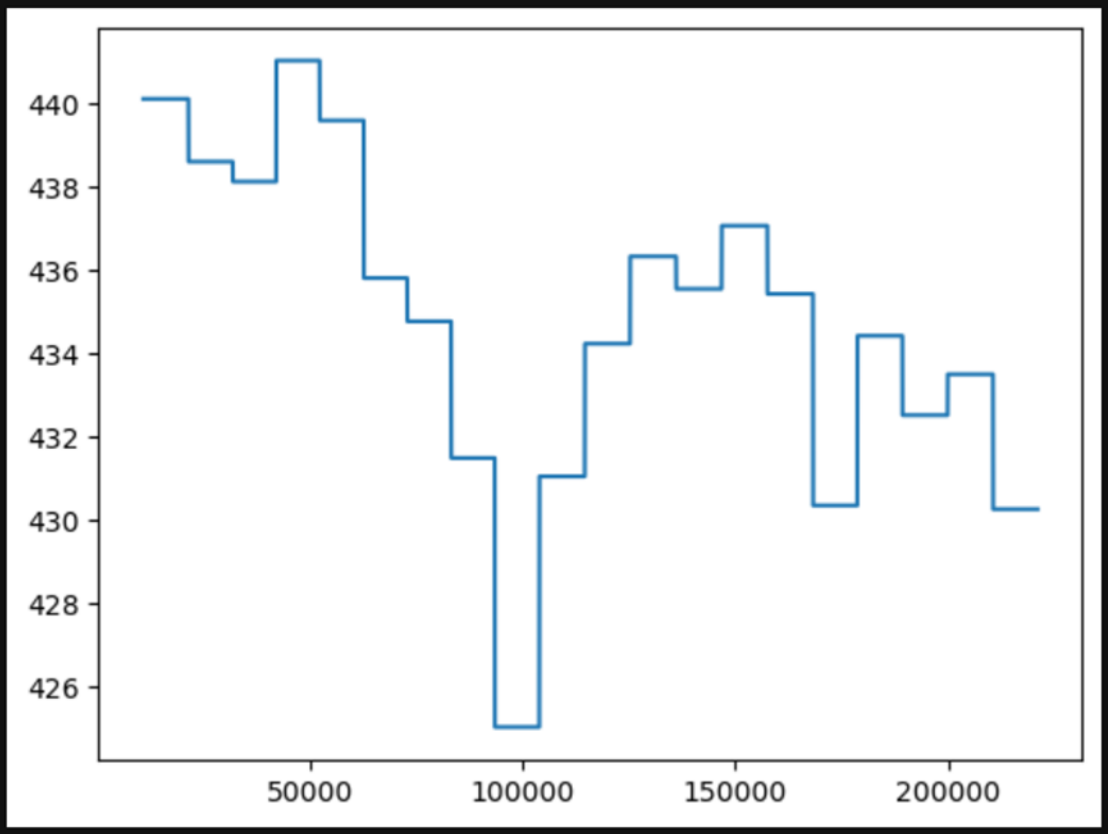


3.2. Stock Price vs Date: Plotting the Stock Price Graph using plot() function of MatPlotLib library.

Plotting the Stock Price Graph

```
[40]: plt.plot(data_till_29.stockPrice)

[40]: [<matplotlib.lines.Line2D at 0x199dfefd000>]
```



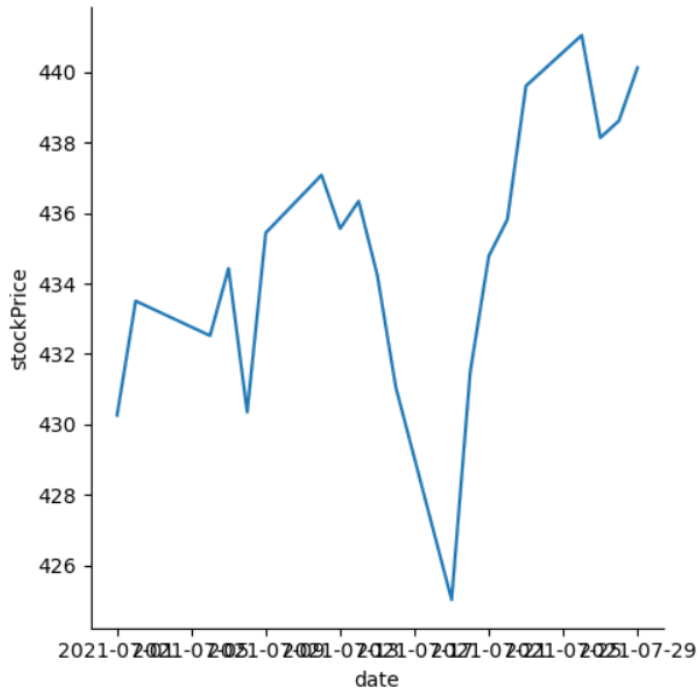
4. Feature Engineering and Data Aggregation

- In this we have to collect the data from the various databases and then we summarize the information as per our need and then we use it for the further process. We can define them using the different graphs. For getting the graph function we need to import the Seaborn library for using the function.

- The Function like boxplot(),barplot(),hist() and many more.

```
[ ] sns.relplot( x="date", y="stockPrice",data=data_till_29,kind='line')
```

```
<seaborn.axisgrid.FacetGrid at 0x199dffb5ff0>
```



- In the code stock price is on y-axis and date on the x-axis as we are seeing the price of the stock from the 1st of july to 29th july to find the value of it and to predict for the july 31st.

5. Data Analysis

5.1 Cleaning and Rolling of Data

- It is a process of examining,cleansing,transforming and scaling data with the aim of finding the useful pattern to generate the valuable insights.
- In the below code we are using the info() to find the information about the data type of the column, memory usage and how many values does the column have.
- As shown in figure 5.2, the data Rolling() function is used to find the minimum value of the data as per the choice.

```
[ ] data_till_29.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 210320 entries, 10726 to 221045
Data columns (total 17 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   date            210320 non-null  datetime64[ns]
 1   adjusted close  210320 non-null  float64
 2   expiration      210320 non-null  object
 3   strike          210320 non-null  float64
 4   call/put        210320 non-null  object
 5   ask             210320 non-null  float64
 6   bid             210320 non-null  float64
 7   mean price      210320 non-null  float64
 8   iv              210320 non-null  float64
 9   volume          210320 non-null  int64
 10  open interest   210320 non-null  int64
 11  stockPrice      210320 non-null  float64
 12  delta           210320 non-null  float64
 13  vega            210320 non-null  float64
 14  gamma           210320 non-null  float64
 15  theta           210320 non-null  float64
 16  rho             210320 non-null  float64
dtypes: datetime64[ns](1), float64(12), int64(2), object(2)
memory usage: 28.9+ MB
```

Figure 5.1

Moving Average of first 100 values in the data

```
1  ma100=data_till_29.stockPrice.rolling(100).mean()
```

```
[ ] 1  print(ma100)
```

Figure 5.2

Here we are getting the first 100 values to find the mean values and to get that 100 values from the data set we are rolling() and doing it with mean().

6. Data Visualization

6.1 Data Representation

- It is a Graphical Representation of the data in which we represent the data into graph,plot,box and many more.
- Here we are representing the first 100 or 200 values from the data set to find the difference by viewing it and the plot() use in it for making them graphically visible.

```
plt.figure(figsize=(12,6))
plt.plot(data_till_29.date,data_till_29.stockPrice,'b')
plt.plot(data_till_29.date,ma100,'r')
plt.plot(data_till_29.date,ma200,'g')
```

```
[<matplotlib.lines.Line2D at 0x199fe816350>]
```

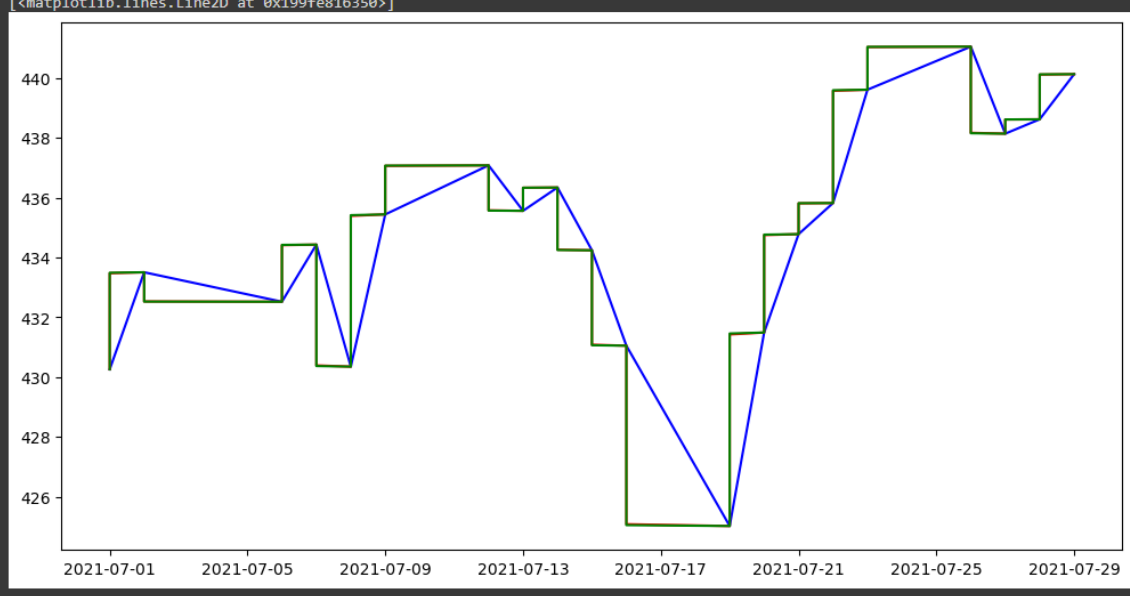


Figure 6.1

6.2 Finding the shape of the dataset

- In this we use the shape() to find the shape of the data which will help us to know of the data.

```
[ ] data_till_29.shape
```

```
(210320, 17)
```

Figure 6.2

7. Model Training

7.1 Importing train_test_split

- For training the data we have to split the data in train and test. This is the most basic and essential part of model training. First, we have to import train_test split from sklearn. The size of the test data is 20 percent and the training is 80 percent.

Splitting the dataset into training and testing

```
# Splitting the dataset into Training and Testing
trainingData=pd.DataFrame(data_till_29['stockPrice'])
testingData=pd.DataFrame(data['stockPrice'][0:10725])
print(trainingData.shape)
print(testingData.shape)
```

```
(210320, 1)
(10725, 1)
```

Figure 7.1

7.2 Applying MinMaxScaler

- The variables that are measured on various scales do not contribute equally to model training and model fitting and might end up creating a bias.
- So, this is why we are using MinMaxScaler before training and fitting the model.

Scaling down the values

```
from sklearn.preprocessing import MinMaxScaler
scaler=MinMaxScaler(feature_range=(0,1))

```

```
training_array=scaler.fit_transform(trainingData)
testing_array=scaler.fit_transform(testingData)

```

Figure 7.2

7.3 Applying LSTM model

- Importing LSTM Neural Network model from keras
- We imported LSTM and sequential models from keras library which will be used to train the data.
- As shown in Figure 7.4, there are five layers created in this LSTM model and we use the "***relu"*** activation function.
- In figure 7.5 the summary of the LSTM model is shown.

Importing LSTM model and sequential model

```
from keras.layers import Dense,Dropout,LSTM
from  keras.models import Sequential
```

Figure 7.3

Creating layers for the LSTM model

```
[ ]
model=Sequential()
model.add(LSTM(units=50,activation='relu',return_sequences=True,input_shape=(x_train.shape[1],1)))
model.add(Dropout(0.2))

model.add(LSTM(units=60,activation='relu',return_sequences=True))
model.add(Dropout(0.3))

model.add(LSTM(units=80,activation='relu',return_sequences=True))
model.add(Dropout(0.4))

model.add(LSTM(units=120,activation='relu'))
model.add(Dropout(0.5))

model.add(Dense(units=1))
```

Figure 7.4

```
model.summary()
```

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm (LSTM)                 (None, 100, 50)           10400

 dropout (Dropout)           (None, 100, 50)           0

 lstm_1 (LSTM)               (None, 100, 60)           26640

 dropout_1 (Dropout)         (None, 100, 60)           0

 lstm_2 (LSTM)               (None, 100, 80)           45120

 dropout_2 (Dropout)         (None, 100, 80)           0

 lstm_3 (LSTM)               (None, 120)               96480

 dropout_3 (Dropout)         (None, 120)               0

 dense (Dense)               (None, 1)                 121

_________________________________________________________________
```

Figure 7.5

- Moreover, we used Adam optimizer to optimize our model and the epochs = 4 for the model.

```
model.compile(optimizer='adam',loss='mean_squared_error')
model.fit(x_train,y_train,epochs=4)
```

Figure 7.6

8. Prediction and Testing

8.1 Merging the columns for testing

- In this step the first 100 columns of training data are merged to test data.

```
past_100_days= trainingData.head(100)
final_df=past_100_days.append(testingData,ignore_index=True)
print(final_df)
```

Figure 8.1

8.2 Scaling the data

- Scaling the test data is necessary so we scale the data in this step.

```
input_data=scaler.fit_transform(final_df)
print(input_data)
```

Figure 8.2

8.3 Making predictions.

- As shown in figure 8.3, the model predicted the value of stock price on 30th date.
- Moreover, the prediction is also shown in visualization form in figure 8.4 for better understanding.
- The model predicted that the value of stock price on the 30th date would be around 438.960.
- Furthermore, in figure 8.5 mean_squared_error is used to predict the accuracy of the model. Accuracy depends on the mean_squared_error value.if the value is lesser then accuracy is more or visa versa.

```
#Making Predictions
y_predicted=model.predict(x_test)
```

```
336/336 [==============================] -
```

```
print(y_predicted)
```

```
[0. 0. 0. ... 0. 0. 0.]
[[-0.00522405]
 [-0.00517016]
 [-0.00501982]
 ...
 [ 0.        ]
 [ 0.        ]
 [ 0.        ]]
```

Figure 8.3

```
plt.figure(figsize=(12,6))
plt.ylim(438.81, 439)
plt.plot(y_predicted,'r+')
plt.plot(testingData,'g')
plt.xlabel('Time')
plt.ylabel('Stock price')
```

```
Text(0, 0.5, 'Stock price')
```

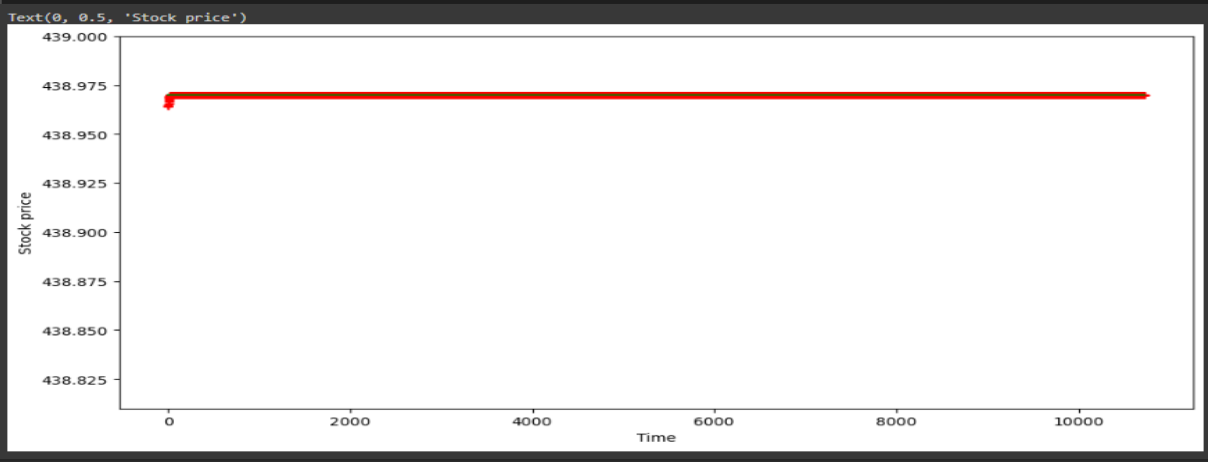


Figure 8.4

```
print(mean_squared_error(original_value,y_predicted))
```

```
2.311852636893832e-08
```

Figure 8.5

9. Validation and matrics (utilization of results)

9.1 Predicted column is added to dataset

- For better understanding the predicted value is concatenated in the dataset as a new feature as shown in figure 9.1 so, It can give better understand of prediction and relation of predicted column.

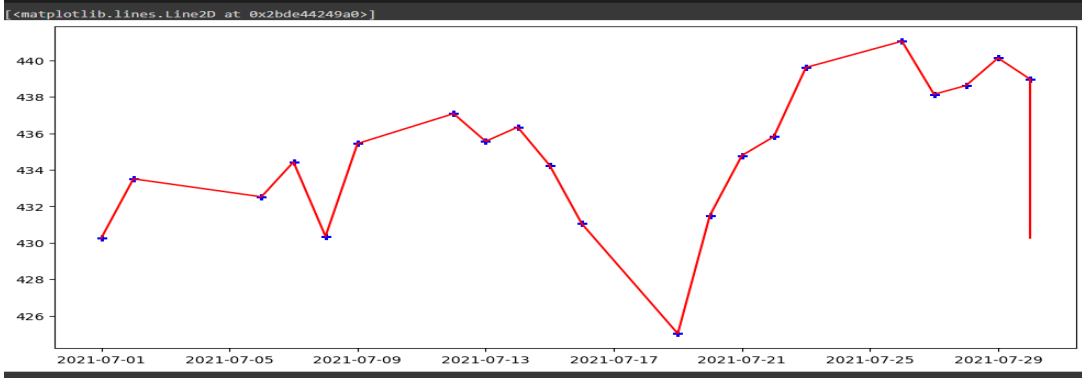
```
[ ]  data["predictedStockPrice"] = pd.Series(stock_price_till_30)
     data.head()
```

	date	adjusted close	expiration	strike	call/put	ask	bid	mean price	iv	volume	open interest	stockPrice	delta	vega	gamma	theta	rho	C	predictedStockPrice
0	2021-07-30	438.51	07/30/2021	215.0	C	224.56	223.35	223.955	-1.0	0	0	438.97	0.0	0.0	0.0	0.0	0.0	430.260000	430.260000
1	2021-07-30	438.51	07/30/2021	215.0	P	0.01	0.00	0.000	-1.0	0	1401	438.97	0.0	0.0	0.0	0.0	0.0	438.963928	438.963928
2	2021-07-30	438.51	07/30/2021	220.0	C	219.56	218.35	218.955	-1.0	0	1	438.97	0.0	0.0	0.0	0.0	0.0	438.963989	438.963989
3	2021-07-30	438.51	07/30/2021	220.0	P	0.01	0.00	0.000	-1.0	50	328	438.97	0.0	0.0	0.0	0.0	0.0	438.964172	438.964172
4	2021-07-30	438.51	07/30/2021	225.0	C	214.56	213.35	213.955	-1.0	0	0	438.97	0.0	0.0	0.0	0.0	0.0	438.964478	438.964478

Figure 9.1

9.2 Visualize the updated dataset

- As shown in figure 9.2, Finally, the graph has been plotted to understand the prediction graphically. So, one can understand the result in a better way by just visualizing the plot.

```
plt.figure(figsize=(12,6))
plt.plot(data.date,data.stockPrice,'b+')
plt.plot(data.date,data.predictedStockPrice,'r')
```

```
[<matplotlib.lines.Line2D at 0x2bde44249a0>]
```